Universidad Rafael Landívar

Facultad de ingeniería

Tecnologías Emergentes e Innovación

Sección 01

Ing. César Rojas

Proyecto de aplicación: Agente de IA especializado en Pensamiento Computacional

Manual técnico

Diego Manuel Rivas Arguijo – 1084522

Katherine Andrea Mayen Rivera – 1129222

Diego Estuardo Azurdia Marín – 1010821

Javier Estuardo Godínez Gudiel – 1179222

Guatemala, 07 de abril de 2026

Índice

Presentación de aplicación 3
Manual técnico 4
2.0. Enlace de repositorio 4
2.1. Diseño y resolución sistemática del problema 4
2.2. Arquitectura del sistema 5
2.3. Flujos de datos 6
2.4. Integración con herramientas de PLN vía API, dependencias e integraciones 8
2.5. Implementación de voz 9
2.6. Proceso de fine-tuning 10
2.7. Tecnologías utilizadas 14
2.8. Diagrama de Gantt de la ejecución 15
2.9. Costos 16
2.10. Prompts 19
2.11. Modelo de datos 20
2.12. Integración con una tecnología emergente adicional 23
2.13. Análisis sistemático e impacto en el entorno 24
2.14. Procesamiento y visualización de datos 26
Mapa del sistema 32
Dataset de fine-tuning 33

Presentación de aplicación

Enlace a aplicación:

tutor-ia-landivar.vercel.app/

Enlace a repositorio:

https://github.com/deam1602/agente-tutor-ia

Enlace a manual de usuario:

https://canva.link/kulr3fnkid3qlgm

Enlace de presentación ejecutiva:

https://canva.link/hszf1wn54933yas

Manual técnico

2.0. Enlace de repositorio

https://github.com/deam1602/agente-tutor-ia

2.1. Diseño y resolución sistemática del problema

Problemática identificada:

El sistema fue diseñado para resolver la necesidad de brindar apoyo académico a estudiantes del curso de Pensamiento Computacional mediante una interfaz conversacional. La solución propuesta organiza el problema en dos módulos principales: un módulo de chat orientado a la asistencia teórica y un módulo de voz orientado a facilitar la interacción multimodal.

Solución propuesta:

La resolución del problema se abordó de forma estructurada. En primer lugar, se definió una ruta de chat encargada de validar los mensajes del usuario, filtrar entradas inválidas, detectar si la consulta pertenece al contexto académico permitido y restringir solicitudes que intenten obtener código completo. Posteriormente, la solicitud válida es enviada al modelo de inteligencia artificial, y la respuesta es revisada nuevamente para evitar que entregue bloques de código o soluciones listas para ejecutar. Este diseño permite mantener el enfoque educativo del sistema y asegurar que la herramienta funcione como apoyo al aprendizaje y no como un generador de respuestas finales.

De forma complementaria, se incorporó una ruta de procesamiento de voz que permite manejar conversiones entre voz y texto. Esta parte amplía la accesibilidad del sistema, aunque en la versión actual su funcionamiento es simulado y representa una base preparada para futuras integraciones con APIs especializadas de reconocimiento y síntesis de voz.

2.2. Arquitectura del sistema

El sistema sigue una arquitectura cliente-servidor integrada mediante el framework Next.js, el cual permite manejar tanto el frontend como el backend dentro de una misma aplicación. El flujo general del sistema es el siguiente:

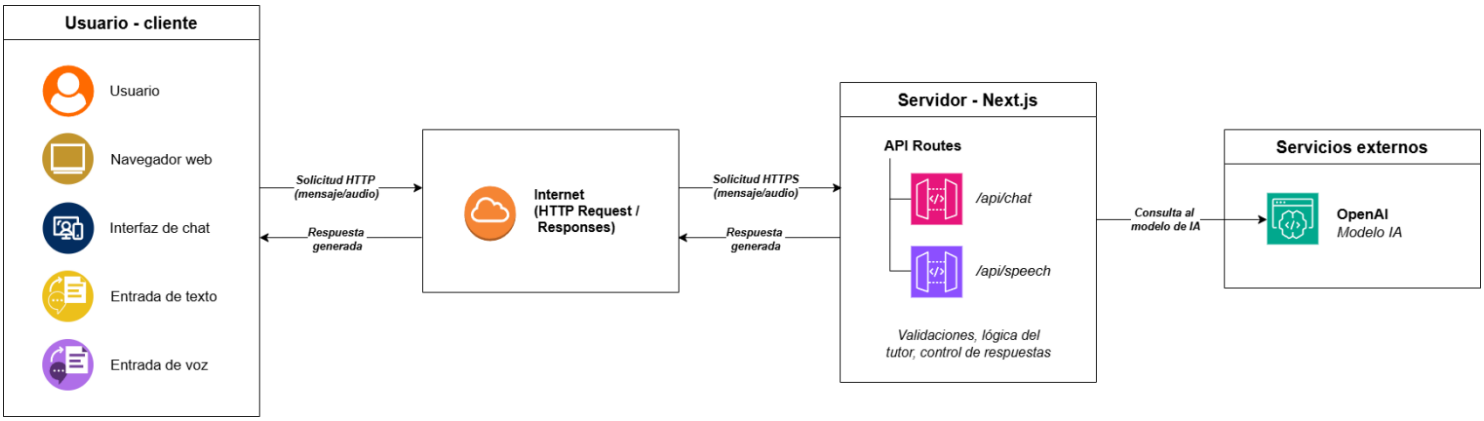
1. El usuario interactúa con la interfaz web mediante el chat
2. La solicitud es enviada a las rutas de API del backend
3. El backend procesa la solicitud y la envía al servicio de inteligencia artificial
4. El modelo genera una respuesta basada en la consulta
5. La respuesta es retornada al frontend y renderizada en la interfaz del chat

Adicionalmente, el sistema cuenta con un conjunto de scripts encargados de la validación de datos y el entrenamiento del modelo, lo cual permite mejorar la calidad de las respuestas generadas. Se optó por un enfoque simplificado que permite una implementación más eficiente y fácil de mantener.

Visualización de arquitectura

El sistema se divide en tres componentes principales: la interfaz de usuario, las rutas de API encargadas del procesamiento y los servicios externos de inteligencia artificial y voz. Esta separación permite mantener una estructura clara, modular y escalable.

En este modelo, el usuario interactúa con una interfaz web que envía solicitudes al servidor, donde se procesan mediante rutas de API y se integran con servicios externos de inteligencia artificial.



2.3. Flujos de datos

Flujo de procesamiento del chat

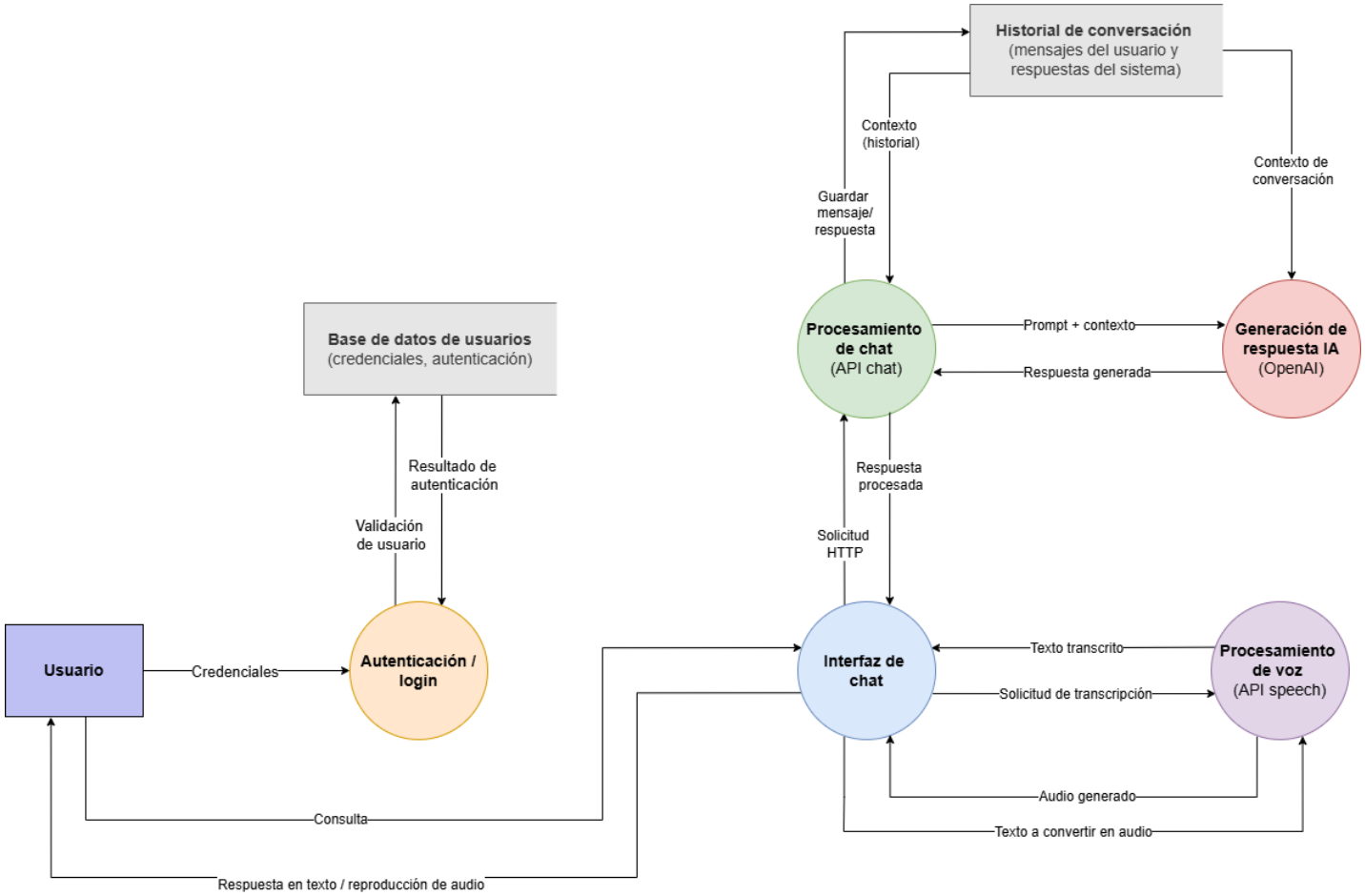
Este diagrama describe el flujo interno del sistema para el procesamiento de mensajes del usuario, incluyendo validaciones, control de contexto académico y generación de respuestas mediante el modelo de inteligencia artificial.

Flujo de procesamiento de datos

El sistema inicia con la autenticación del usuario, donde las credenciales ingresadas son validadas contra la base de datos de usuarios. Una vez autenticado, el usuario puede interactuar con el tutor mediante texto o voz.

En caso de utilizar voz, el audio es procesado por el módulo de voz (speech-to-text), convirtiéndolo en texto que luego es enviado como una consulta normal. Luego el mensaje es enviado al módulo de procesamiento de chat, donde se consulta el historial de conversaciones para obtener el contexto necesario.

Con esta información se construye una solicitud que es enviada al modelo de IA, el cual genera una respuesta. Dicha respuesta es almacenada junto con el mensaje del usuario en el historial de conversaciones y enviada a la interfaz para su visualización. El sistema permite convertir las respuestas en audio mediante text-to-speech, permitiendo al usuario escuchar la información generada.



2.4. Integración con herramientas de PLN vía API, dependencias e integraciones vía API

El sistema integra herramientas de Procesamiento de Lenguaje Natural (PLN) mediante el uso de APIs externas, específicamente a través del servicio de inteligencia artificial proporcionado por OpenAI.

La comunicación con este servicio se realiza desde el backend de la aplicación, implementado en Next.js, a través de la ruta /api/chat. En esta ruta, los mensajes del usuario son procesados, validados y estructurados junto con el contexto de la conversación antes de ser enviados al modelo de inteligencia artificial.

La integración se realiza utilizando el cliente oficial de OpenAI, configurado mediante una clave de API almacenada en las variables de entorno, lo que permite mantener la seguridad y evitar la exposición de credenciales sensibles.

El sistema incorpora una serie de validaciones y reglas antes de realizar la consulta al modelo, tales como la verificación del contexto académico, la detección de solicitudes de código completo y el manejo de continuaciones de conversación, garantizando que las respuestas generadas mantengan un enfoque educativo y alineado con los objetivos.

Dependencias e integraciones:

Para la implementación de la integración con el modelo de PLN, el sistema utiliza las siguientes dependencias principales:

- **OpenAI SDK:** integración principal de PLN, utilizada para realizar solicitudes al modelo de inteligencia artificial y procesar las respuestas generadas.
- **Next.js API Routes:** permiten manejar la lógica del backend y actuar como intermediario entre el cliente y los servicios externos.
- **Variables de entorno (.env):** utilizadas para almacenar de forma segura la clave de API y la configuración del modelo.

El flujo de integración consiste en que el frontend envía una solicitud al backend, el cual construye un prompt estructurado y lo envía al servicio de OpenAI, luegp la respuesta generada es recibida, procesada y retornada al cliente.

2.5. Implementación de voz

El sistema incorpora un módulo de interacción por voz que permite mejorar la accesibilidad y la experiencia del usuario mediante el uso de funcionalidades de reconocimiento e integración de audio.

Esta implementación se realiza a través de una ruta de API independiente en el backend, /api/speech, la cual gestiona dos tipos de operaciones:

- **Conversión de voz a texto**: el usuario puede proporcionar una entrada de audio que es enviada al backend, donde se procesa y se transforma en texto. Este texto es posteriormente mostrado en la interfaz de chat, permitiendo al usuario enviarlo como una consulta normal al sistema.
- **Conversión texto a voz**: permite convertir las respuestas generadas por el asistente en audio. Esta opción es activada por el usuario mediante la interfaz, permitiendo escuchar las respuestas en lugar de leerlas.

De esta manera, la implementación de voz actúa como un componente complementario al chat, facilitando una interacción multimodal entre el usuario y el sistema.

```
.env.local    route.ts ...\chat M    route.ts ...\speech
app > api > speech > route.ts > ...
1   import { NextResponse } from 'next/server';
2
3   export async function POST(req: Request) {
4     try {
5       const body = await req.json();
6       const { audioData, type } = body;
7
8       if (!audioData) {
9         return NextResponse.json(
10          { error: 'No se recibió ningún dato de audio/texto' },
11          { status: 400 }
12        );
13      }
14
15      if (type === 'speech-to-text') {
16        /*
17          Procesar el audio recibido y convertirlo a texto interactuando con Whisper API,
18          Google Speech API  o alguna así
19        */
20
21        return NextResponse.json({
22          text: 'Transcripción de prueba del audio (Speech-to-Text simulado).'
23        });
24
25      } else if (type === 'text-to-speech') {
26        /*
27            Procesar texto y convertirlo en stream de audio
28        */
29
30        return NextResponse.json({
31          success: true,
32          message: 'Audio generado simulado (Text-to-Speech simulado).'
33        });
34      }
35
36      return NextResponse.json(
37        { error: 'Tipo de operación no válida. Use speech-to-text o text-to-speech' },
38        { status: 400 }
39      );
40
41    } catch (error) {
42      console.error('Error en /api/speech:', error);
43      return NextResponse.json(
44        { error: 'Error interno en el servidor de voz.' },
45        { status: 500 }
46      );
47    }
48  }
49
```

2.6. Proceso de fine-tuning

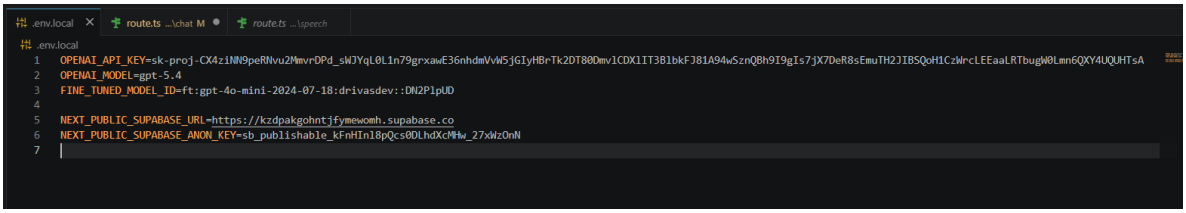
Se implementó fine-tuning con el objetivo de adaptar el modelo a un contexto académico específico, enfocado en el curso de Pensamiento Computacional.

Este proceso inicia con la preparación de un dataset estructurado en formato JSONL, el cual contiene ejemplos de conversaciones entre usuario y asistente. Cada entrada incluye un conjunto de mensajes con roles definidos (user y assistant), lo que permite entrenar al modelo en un formato conversacional. Previo al entrenamiento, el dataset es validado mediante un script automatizado que verifica la correcta estructura de los datos, asegurando que cada registro contenga mensajes válidos con los campos requeridos.

Una vez validado, el dataset es cargado al servicio de OpenAI como archivos de entrenamiento y validación, y posteriormente se crea un proceso de fine-tuning utilizando un modelo base (gpt-4o-mini), al cual se le proporcionan ambos conjuntos de datos para ajustar su comportamiento.

Una vez finalizado el entrenamiento, el modelo ajustado es integrado en el sistema mediante variables de entorno (.env.local), permitiendo que la ruta /api/chat utilice el modelo de fine-tuned en lugar de un modelo base, mejorando la calidad de las respuestas y asegurando que mantengan un enfoque educativo respetando las restricciones del sistema.

Durante el proceso de fine-tuning, el dataset fue dividido en dos conjuntos: un conjunto de entrenamiento (training dataset) y un conjunto de validación (validation dataset). El primero es utilizado por el modelo para aprender patrones y generar respuestas, mientras que el segundo permite evaluar el desempeño del modelo con datos que no han sido utilizados durante el entrenamiento.



El sistema incluye scripts automatizados que permiten ejecutar el flujo de fine-tuning de manera controlada, incluyendo la subida de archivos, la creación del job de entrenamiento y la verificación de su estado.

1. *Validate-dataset.js*

En primer lugar, se creó un script de validación del dataset, el cual lee los archivos en formato JSONL línea por línea, convirtiendo cada registro a objeto y verificando que contenga un arreglo de mensajes válido. Este script comprueba que cada mensaje incluya los campos role y content como cadenas de texto, y que exista al menos una interacción completa entre usuario y asistente. Además reporta errores indicando la línea exacta donde ocurre el problema y muestra la cantidad de ejemplos válidos procesados.

.env.local | route.ts M | validate-dataset.js

scripts > validate-dataset.js > validateFile > lines.forEach() callback

```javascript
const fs = require("fs");
const path = require("path");

function validateFile(filePath) {
  const fullPath = path.join(process.cwd(), filePath);
  const lines = fs.readFileSync(fullPath, "utf-8").split("\n").filter(Boolean);

  let validCount = 0;

  lines.forEach((line, index) => {
    try {
      const obj = JSON.parse(line);

      if (!obj.messages || !Array.isArray(obj.messages) || obj.messages.length < 2) {
        throw new Error("El campo messages no es válido.");
      }

      for (const msg of obj.messages) {
        if (
          !msg.role ||
          typeof msg.role !== "string" ||
          !msg.content ||
          typeof msg.content !== "string"
        ) {
          throw new Error("Cada mensaje debe tener role y content tipo string.");
        }
      }

      validCount++;
    } catch (err) {
      console.error(`Error en línea ${index + 1}: ${err.message}`);
    }
  });

  console.log(`Archivo revisado: ${filePath}`);
  console.log(`Ejemplos válidos: ${validCount}`);
}

validateFile("../app/data/pensamiento_computacional_train.jsonl");
validateFile("../app/data/pensamiento_computacional_valid.jsonl");
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\kathe\OneDrive\Escritorio\agente-tutor-ia\scripts> node .\validate-dataset.js
Archivo revisado: ../app/data/pensamiento_computacional_train.jsonl
Ejemplos válidos: 140
Archivo revisado: ../app/data/pensamiento_computacional_valid.jsonl
Ejemplos válidos: 57
PS C:\Users\kathe\OneDrive\Escritorio\agente-tutor-ia\scripts>
```

2. *Create-finetune.js*

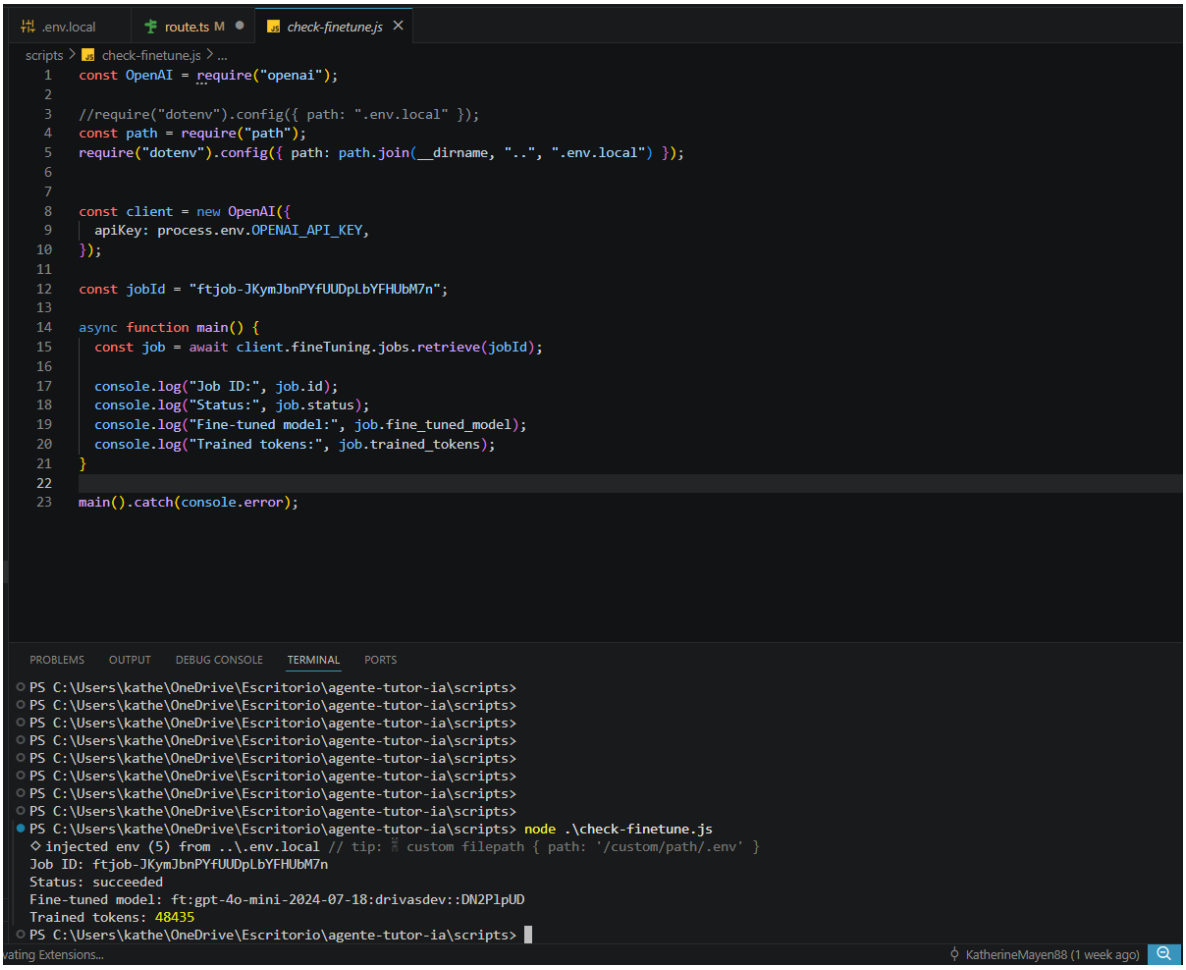
En segundo lugar, se implementó un script de creación del fine-tuning, el cual se encarga de subir los archivos de entrenamiento y validación a la API de OpenAI utilizando streams de archivos. Una vez cargados, el script crea un job de fine-tuning especificando el modelo base a utilizar y los identificadores de los archivos previamente subidos, iniciando así el proceso de entrenamiento del modelo ajustado.

```
.env.local    route.ts M    create-finetune.js
scripts > create-finetune.js > ...
1  const OpenAI = require("openai");
2  const fs = require("fs");
3
4  //require("dotenv").config({ path: ".env.local" });
5  const path = require("path");
6  require("dotenv").config({ path: path.join(__dirname, "..", ".env.local") });
7
8  const client = new OpenAI({
9    apiKey: process.env.OPENAI_API_KEY,
10 });
11
12 async function main() {
13   const trainingFile = await client.files.create({
14     file: fs.createReadStream("../app/data/pensamiento_computacional_train.jsonl"),
15     purpose: "fine-tune",
16   });
17
18   console.log("Training file subido:", trainingFile.id);
19
20   const validationFile = await client.files.create({
21     file: fs.createReadStream("../app/data/pensamiento_computacional_valid.jsonl"),
22     purpose: "fine-tune",
23   });
24
25   console.log("Validation file subido:", validationFile.id);
26
27   const job = await client.fineTuning.jobs.create({
28     training_file: trainingFile.id,
29     validation_file: validationFile.id,
30     model: "gpt-4o-mini-2024-07-18",
31   });
32
33   console.log("Fine-tuning job creado:");
34   console.log("Job ID:", job.id);
35   console.log("Status:", job.status);
36 }
37
38 main().catch((err) => {
39   console.error("Error creando fine-tuning:", err);
40 });
```

```
PS C:\Users\kathe\OneDrive\Escritorio\agente-tutor-ia\scripts> node .\create-finetune.js
[dotenv@17.3.1] injecting env (5) from ..\.env.local -- tip: 🛠 run anywhere with `dotenvx run -- yourcommand`
Training file subido: file-L2Un2ifs7Kmtf54c5dJCrZ
Validation file subido: file-PXY5pPbgKA1rHhF2aebxBr
Fine-tuning job creado:
Job ID: ftjob-JKymJbnPYfUUDpLbYFHUbM7n
Status: validating_files
PS C:\Users\kathe\OneDrive\Escritorio\agente-tutor-ia\scripts>
```

3. *Check-finetune.js*

Finalmente, se desarrolló un script de verificación que permite consultar el estado del proceso de fine-tuning mediante el identificador del job. Este script obtiene información como el estado actual del entrenamiento, el modelo generado y la cantidad de tokens utilizados, lo que permite monitorear el progreso y confirmar la finalización del proceso.



2.7. Tecnologías utilizadas

El sistema fue desarrollado utilizando tecnologías orientadas al desarrollo web, procesamiento de lenguaje natural e integración con servicios en la nube.

- **Next.js:** Para desarrollo fullstack, en el frontend y backend se utilizó Next.js permitiendo implementar una arquitectura fullstack en una sola aplicación, facilitando la creación de la interfaz de usuario y las rutas de API para el procesamiento del chat y la voz.

- **OpenAI API:** Para el procesamiento de lenguaje natural y generación de respuestas, en la generación de respuestas se integró la API de OpenAI, utilizada como motor de procesamiento de lenguaje natural (PLN).

- **Fine-tuning:** Se implementó un proceso de fine-tuning para adaptar el comportamiento del modelo al contexto académico del sistema.

- **Supabase:** Para la base de datos y autenticación de usuarios. La persistencia de datos y autenticación de usuarios utilizaron Supabase, permitiendo gestionar el acceso al sistema y almacenar el historial de conversaciones de los usuarios y sus credenciales.

- **Vercel:** El despliegue y hosting de la aplicación se realizó mediante Vercel, facilitando la publicación de aplicaciones Next.js y su disponibilidad en la web.

- **Git:** Para el control de versiones y colaboración, permitiendo gestionar cambios en el código fuente y mantener un historial del desarrollo del proyecto.

- **Node.js:** Para la ejecución de scripts y manejo del entorno de desarrollo.

- **JavaScript / TypeScript**: Como lenguaje principal de desarrollo.

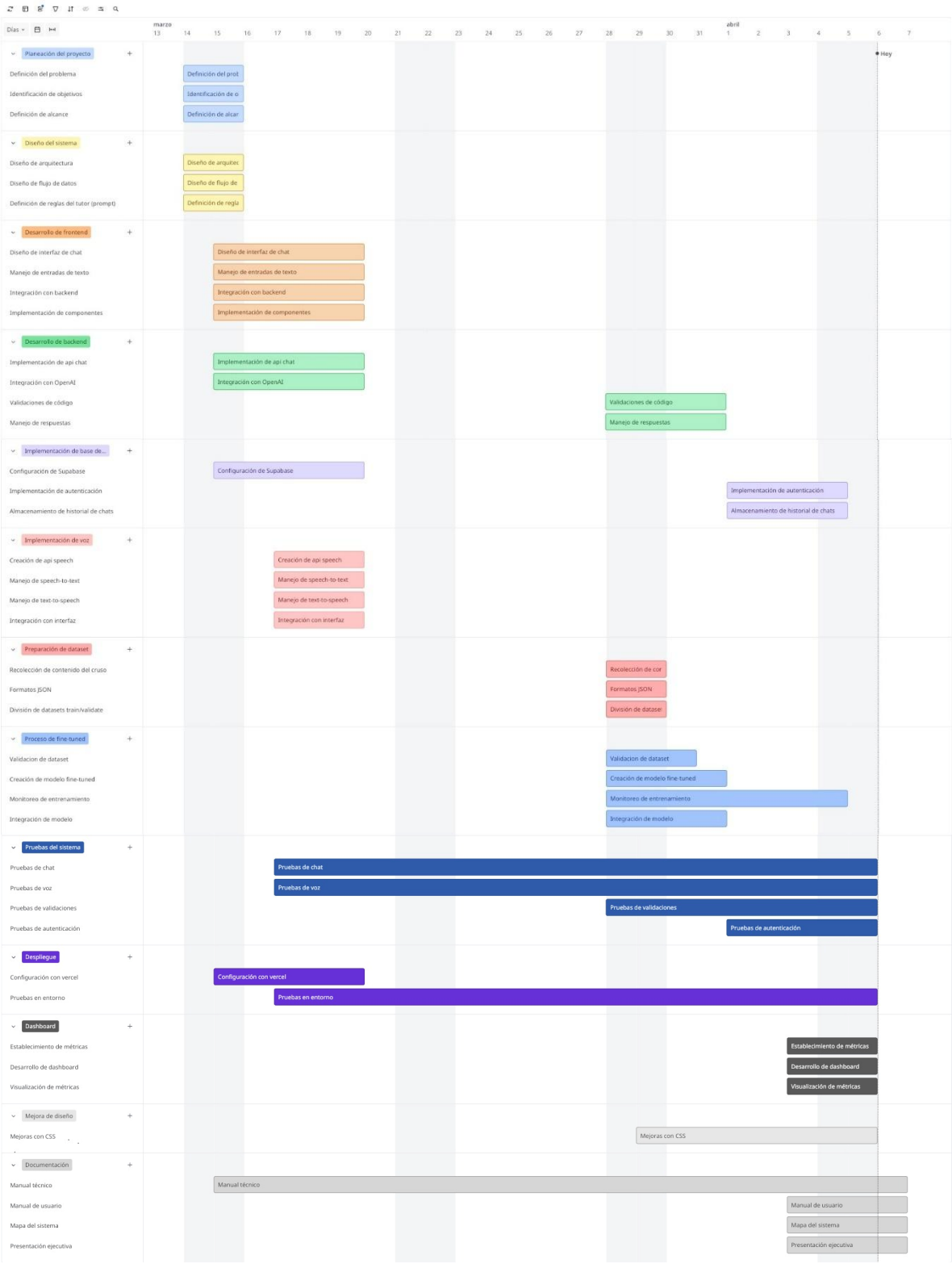
- **Dotenv**: Para la gestión de variables de entorno y configuración segura del sistema.

- **Recharts:** Para la visualización de métricas en el dashboard se utilizó la librería Recharts, la cual permite representar datos mediante gráficas interactivas como diagramas de barras, áreas y líneas. El sistema está organizado bajo una estructura modular propia de Next.js, utilizando carpetas como app, components, lib y scripts, lo que facilita el mantenimiento, la escalabilidad y la separación de responsabilidades.

2.8. Diagrama de Gantt de la ejecución

Enlace de diagrama en miro:

https://miro.com/app/board/uXjVGnSH310=/?share_link_id=483875387123



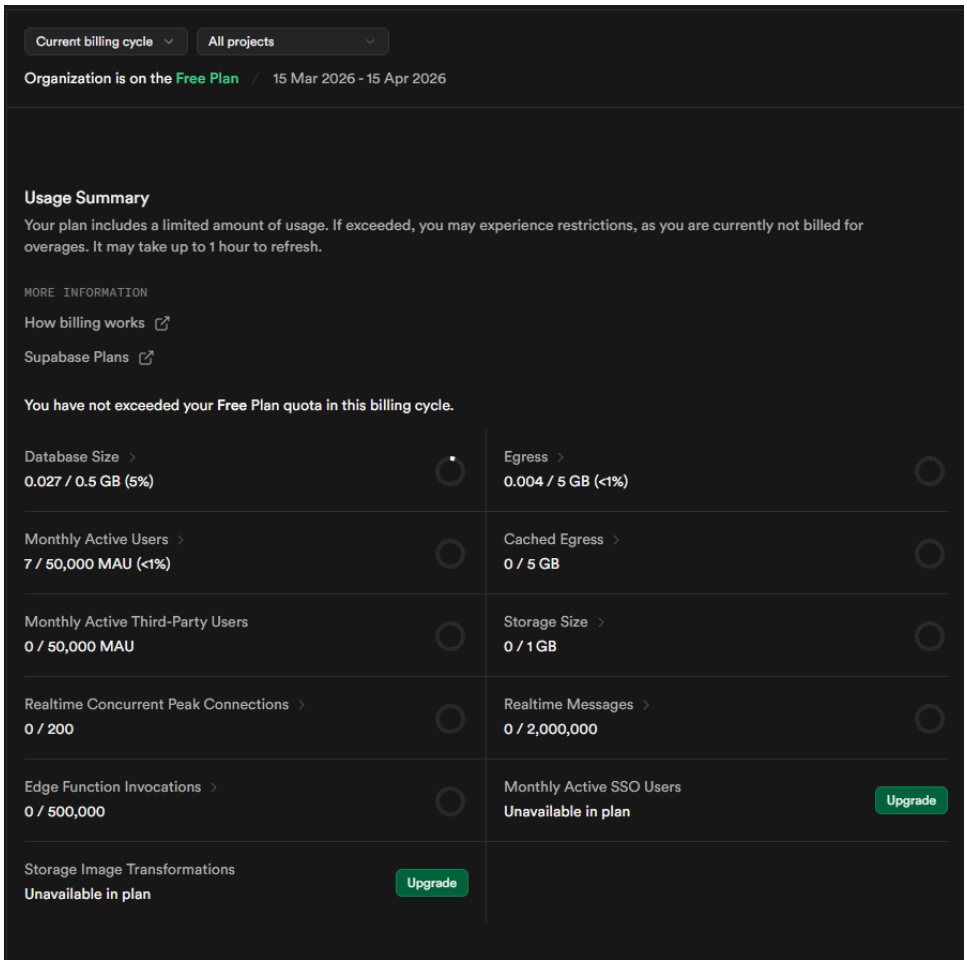
2.9. Costos

El modelo económico del proyecto se divide en tres puntos tecnológicos: la persistencia de datos, el alojamiento en la red, y el motor de procesamiento de Lenguaje Natural.

1. Persistencia de datos y autenticación

El ecosistema para validación de perfiles, permisos y mantenimiento del historial del chat está encargado a BaaS Supabase, utilizado actualmente en su capa gratuita. Hasta el cierre de la fase actual, el tamaño de la base de datos es de apenas 25.94 MB de un límite disponible de 500 MB. El almacenamiento de conversaciones se compone puramente de texto plano, haciendo que los registros sean en extremo ligeros.

Bajo estas métricas, se podría almacenar miles de conversaciones durante semestres enteros de uso ininterrumpido sin llegar jamás al límite, garantizando operatividad integral a costo $0.00 en bases de datos. Además, el tráfico de red ha consumido, 0.004 GB al no almacenar imágenes ni archivos multimedia pesados.



2. *Distribución y alojamiento web (Vercel)*

La aplicación construida con Next.js se encuentra desplegada en la red de borde de distribución Vercel. Gracias a las optimizaciones de Next.js, la carga de activos requerida por los navegadores de los usuarios es muy baja.

Durante el ciclo intenso de pruebas, la transferencia rápida de datos consumió únicamente 35.32 MB frente a un límite de 100 GB. En un escenario de carga académica masiva, el sistema se mantendrá dentro de este límite. El costo por concepto de servidores y alojamiento se proyecta fijamente en $0.00.

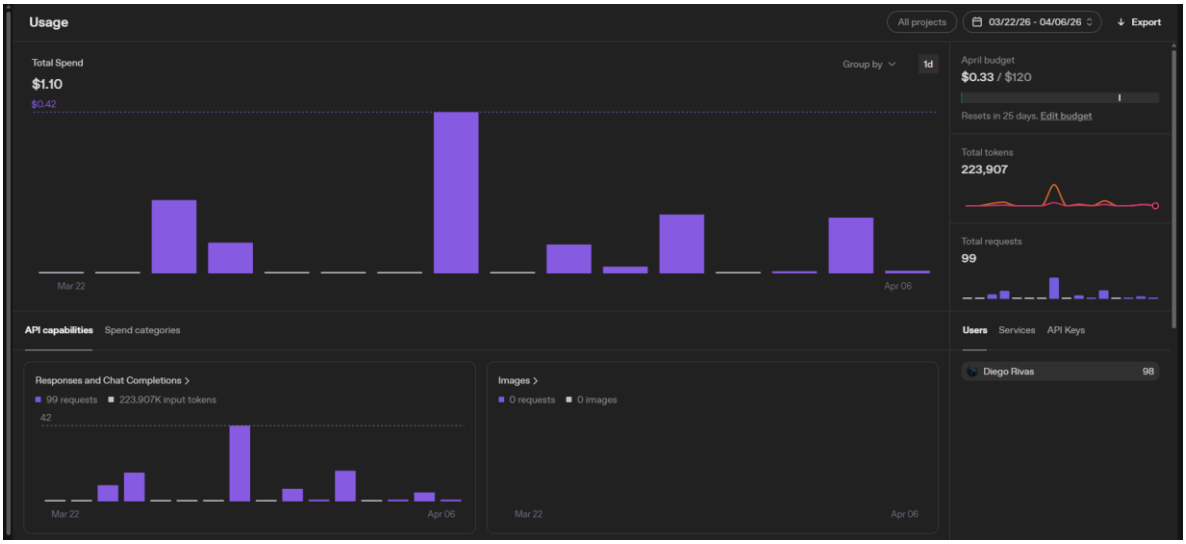
3. *Inteligencia Artificial*

El único gasto operativo directo que la plataforma genera lo tiene la API de OpenAI, la cual responde en arquitectura transaccional de Pay As You Go. Revisando los monitores del proveedor directo durante el proyecto, tenemos el procesamiento de 223,907 tokens evaluados a lo largo de 99 peticiones conversacionales.

El gasto total facturado por todos estos volúmenes ha sido de $1.10 USD. Cada interacción académica larga tiene un costo neto para la nube de $0.011 USD.

Para demostrar la viabilidad, proyectemos la siguiente estimación semestral para un bloque académico:

- **Carga estudiantil:** única sección de 50 alumnos de Pensamiento Computacional.
- **Tasa de consultas asíncronas:** 20 preguntas avanzadas por alumno al ciclo = 1,000 Peticiones.
- El costo acumulado al término del semestre por solventar las dudas de toda la clase rondaría los $11.00 USD aproximadamente.



La arquitectura del sistema es técnica y financieramente estable. Operamos servicios empresariales modernos manteniéndonos libres de costos al trabajar arquitecturas puras de texto y consultas indexadas. Por una inversión inferior a los $15 USD semestrales, es factible modernizar el apoyo pedagógico de un curso universitario a través de nuestro agente las 24 horas del día.

2.10. Prompts

El prompt principal define al asistente como un tutor académico del curso de Pensamiento Computacional, estableciendo su rol, alcance y restricciones. Este prompt limita las respuestas a temas relacionados con programación básica, algoritmos y lógica computacional, evitando que el sistema responda consultas fuera de este contexto.

Se establecen reglas importantes que prohíben la generación de código completo o soluciones listas para ejecutar, promoviendo en su lugar explicaciones conceptuales, pasos lógicos y pseudocódigo.

El sistema utiliza prompts dinámicos que se generan según el contexto de la conversación, por ejemplo, cuando el usuario solicita continuar una explicación o cambiar el nivel de detalle, permitiendo adaptar las respuestas del modelo de forma más precisa.

```
19  const tutorPrompt = `
20  Eres un tutor académico del curso de Pensamiento Computacional.
21
22  Tu alcance está limitado únicamente a temas del curso, por ejemplo:
23  - programación
24  - algoritmos
25  - pseudocódigo
26  - variables
27  - tipos de datos
28  - instrucciones
29  - condicionales
30  - ciclos
31  - funciones
32  - listas, arreglos o matrices
33  - validación de datos
34  - lógica de programación
35  - interpretación de código
36  - depuración básica
37
38  REGLAS ESTRICTAS:
39  1. SOLO puedes responder temas y preguntas del curso de Pensamiento Computacional (programación básica, lógica, algoritmos, etc).
40
41  2. Si la pregunta no pertenece al curso, debes rechazarla amablemente e indicar que solo puedes ayudar con temas de Pensamiento Computacional.
42
43  3. NO debes proporcionar código completo ni bloques de código funcionales listos para ejecutar.
44
45  4. NO debes resolver tareas completas.
46
47  5. Si el usuario responde con frases como "sí", "si", "ok", "dale", "explícalo", "continúa", "hazlo", debes asumir que se refiere al último tema discutido y continuar la explicación sin cambiar de contexto.
48
49  6. Cuando el usuario muestre código:
50     - debes explicar el error
51     - debes describir cómo corregirlo
52     - debes usar pseudocódigo o explicación en palabras
53
54  7. SOLO puedes mostrar código en casos MUY puntuales:
55     - como máximo UNA sola línea corta
56     - solo si es indispensable para explicar un símbolo o sintaxis
57     - nunca debes mostrar un bloque de varias líneas
58
59  8. PROHIBIDO:
60     - escribir programas completos
61     - reconstruir el código del usuario corregido
62     - dar soluciones listas para ejecutar
```

```
64  9. SIEMPRE prioriza:
65     - explicación
66     - pasos
67     - lógica
68     - pseudocódigo
69
70  10. SÍ puedes:
71     - explicar conceptos
72     - orientar paso a paso
73     - describir la lógica
74     - dar pseudocódigo
75
76  11. Si el usuario pide código completo, debes rechazarlo amablemente y ofrecer:
77     - explicación paso a paso
78     - pseudocódigo
79     - estructura general
80     - revisión de un fragmento que el estudiante ya tenga
81
82  Ejemplo correcto:
83  "Debes agregar dos puntos al final del if"
84
85  Ejemplo incorrecto:
86  (mostrar todo el bloque corregido)
87
88  Tu objetivo es guiar al estudiante, no resolverle el ejercicio. Tu tono debe ser claro, académico y útil.
89
90  Responde en texto claro, sin usar markdown, sin encabezados con # y sin negritas con *.
91  `;
92
```

2.11. Modelo de datos

El sistema utiliza un modelo de datos relacional implementado en Supabase, diseñado para gestionar usuarios, sesiones de conversación y mensajes intercambiados en el chat. El modelo se compone de tres entidades principales: profiles, chat_sessions y chat_messages, las cuales se relacionan entre sí para mantener la integridad y organización de la información.

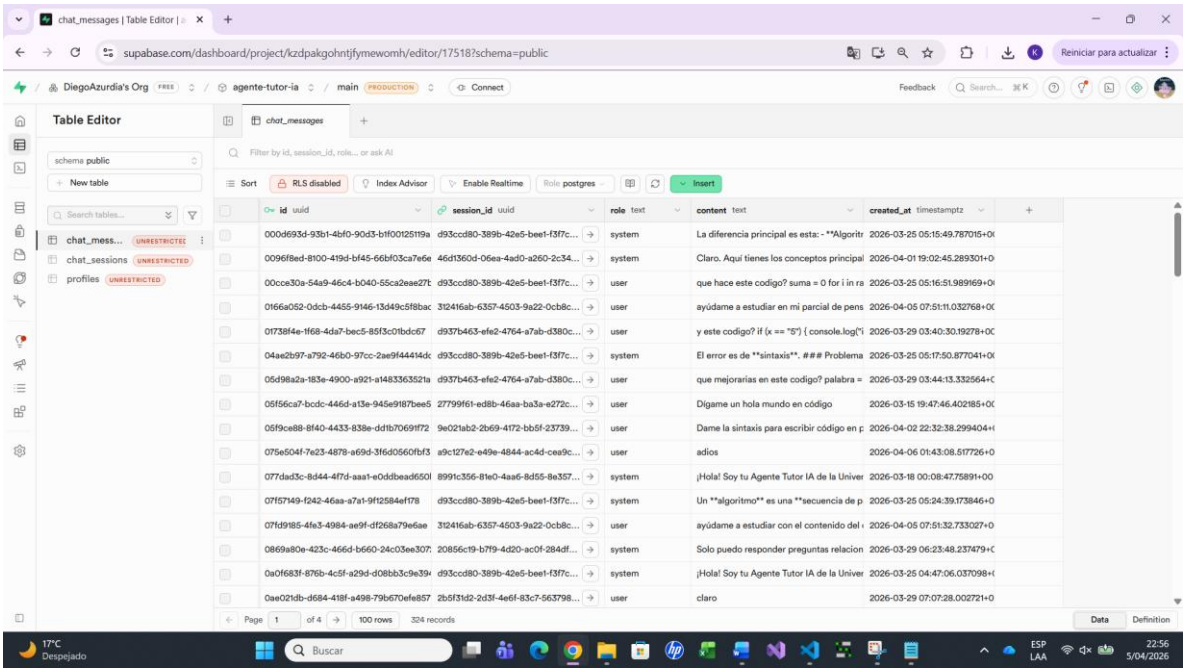
Las tablas profiles, chat_sessions y chat_messages no solo permiten gestionar la autenticación de usuarios y el historial de conversaciones, sino que también funcionan como fuente de datos para el dashboard analítico del sistema, permitiendo generar métricas, reportes y visualizaciones sobre el uso de la plataforma.

Chat_messages (Mensajes)

Esta tabla almacena los mensajes intercambiados en cada conversación. Esta entidad permite mantener el historial completo de la conversación, incluyendo tanto las consultas del usuario como las respuestas generadas por el sistema.

Campos principales:

- **id:** identificador del mensaje.
- **session_id:** referencia a la sesión de conversación.
- **role:** indica si el mensaje es del usuario o del sistema.
- **content:** contenido del mensaje.
- **created_at:** fecha de envío del mensaje.

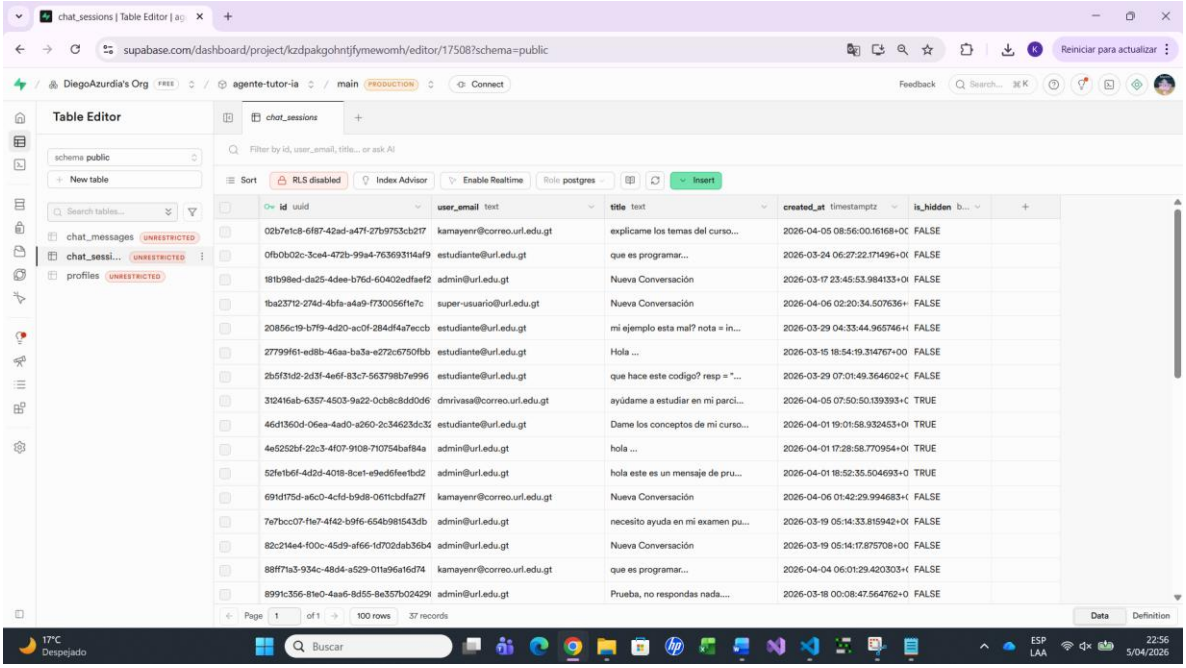


Chat_sessions (Sesiones de conversación)

Esta tabla representa cada conversación iniciada por un usuario. Cada registro en esta tabla corresponde a una conversación independiente dentro del sistema.

Campos principales:

- **id:** identificador único de la sesión.
- **user_email:** correo del usuario que creó la sesión.
- **title:** título de la conversación.
- **created_at:** fecha de creación.
- **is_hidden:** indica si la conversación está oculta.

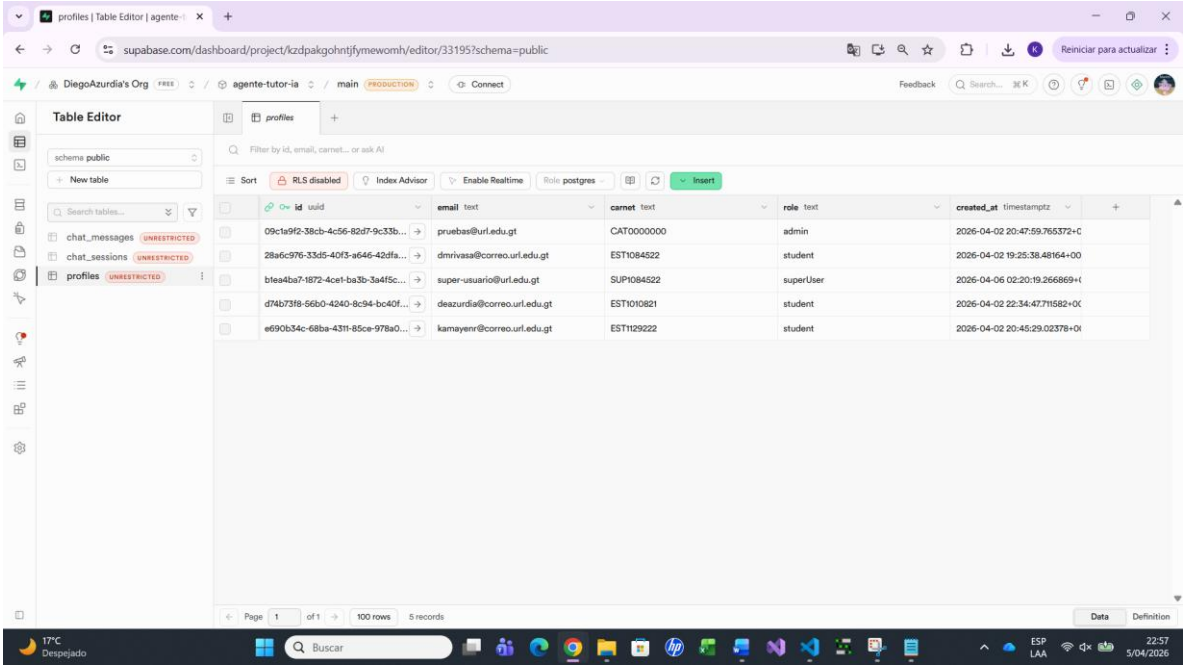


Profiles (Usuarios)

Esta tabla almacena la información de los usuarios del sistema. Esta entidad es utilizada para gestionar la autenticación y control de acceso al sistema.

Campos principales:

- **id:** identificador único del usuario.
- **email:** correo electrónico del usuario.
- **carnet:** identificador académico.
- **role:** rol del usuario (por ejemplo: student, admin, superUser).
- **created_at:** fecha de creación del registro.



Relaciones del modelo:

- Un usuario (profiles) puede tener múltiples sesiones (chat_sessions).
- Una sesión (chat_sessions) puede contener múltiples mensajes (chat_messages).
- Cada mensaje (chat_messages) pertenece a una única sesión mediante el campo session_id.

2.12. Integración con una tecnología emergente adicional

Además del uso de procesamiento de lenguaje natural mediante modelos de inteligencia artificial, el sistema integra una tecnología emergente adicional orientada a la interacción multimodal, que es el módulo de voz. Esta funcionalidad permite ampliar la forma en que el usuario se comunica con la plataforma, incorporando tanto entrada por voz como salida en audio.

La implementación de esta tecnología se realiza a través de una ruta específica en el backend, /api/speech, diseñada para manejar procesos de conversión de voz a texto y de texto a voz. Gracias a esto, el usuario no solo puede escribir sus consultas, sino también dictarlas y escuchar las respuestas generadas por el sistema, lo que mejora la accesibilidad y la experiencia de uso.

El proyecto incorpora como componente innovador el ajuste especializado del modelo mediante fine-tuning, lo que permite adaptar el comportamiento de la inteligencia artificial a un dominio académico concreto. Esto representa una aplicación de tecnologías emergentes dentro del ámbito educativo al combinar modelos generativos, personalización del comportamiento del asistente e interacción por voz en una sola solución.

2.13. Análisis sistemático e impacto en el entorno

Actores

- **Estudiantes:** usuarios principales que utilizan el sistema para resolver dudas y reforzar su aprendizaje.
- **Administradores:** responsables de gestionar usuarios y supervisar el uso del sistema.
- **Sistema de inteligencia artificial:** encargado de procesar las consultas y generar respuestas.
- **Infraestructura tecnológica:** plataformas como Supabase, Vercel y OpenAI que permiten el funcionamiento del sistema.

Procesos impactados

- Resolución de dudas en tiempo real
- Apoyo en el estudio de conceptos de programación
- Interpretación y análisis de código
- Acceso a tutoría académica sin limitaciones de horario
- Optimización de acceso a la información al reducir dependencias de recursos tradicionales o disponibilidad de docentes

Beneficios

- Acceso inmediato a apoyo académico, los estudiantes pueden obtener ayuda en cualquier momento.
- Aprendizaje guiado, el sistema promueve la comprensión mediante explicaciones en lugar de soluciones directas.
- Personalización, el uso de historial permite mantener contexto en las conversaciones.
- Interacción multimodal, permite el uso de texto y voz.
- Escalabilidad, el sistema puede atender múltiples usuarios simultáneamente.
- La incorporación del dashboard analítico permite obtener evidencia del uso real de la plataforma, facilitando la identificación de patrones de comportamiento, consultas frecuentes y oportunidades de mejora en el proceso de aprendizaje.

Riesgos técnicos

- Dependencia de servicios externos como OpenAI
- Posibles errores o imprecisiones en las respuestas generadas
- Posibles fallos en la conectividad o disponibilidad del sistema

Riesgos éticos

- Dependencia excesiva del asistente por parte del estudiante
- Posible generación de respuestas incorrectas que afecten el aprendizaje
- Manejo de datos personales (usuarios y conversaciones), que requiere garantizar privacidad y seguridad

Análisis de sostenibilidad

El sistema presenta una buena sostenibilidad a nivel tecnológico debido al uso de herramientas modernas y ampliamente adoptadas como Next.js, Supabase y Vercel.

Su arquitectura modular permite realizar mejoras y actualizaciones sin afectar el funcionamiento global, pero su sostenibilidad depende en parte del uso de servicios externos como OpenAI, lo que implica considerar costos asociados y disponibilidad del servicio a largo plazo.

Análisis de escalabilidad

El sistema está diseñado para escalar de manera eficiente, ya que utiliza tecnologías en la nube que permiten manejar múltiples usuarios simultáneamente. Plataformas como Vercel y Supabase facilitan la escalabilidad automática, mientras que el uso de APIs permite distribuir la carga de procesamiento.

La arquitectura modular del sistema permite integrar nuevos servicios, mejorar el modelo de inteligencia artificial o ampliar funcionalidades sin necesidad de rediseñar completamente la solución.

2.14. Procesamiento y visualización de datos

El sistema incorpora una interfaz interactiva de analítica que permite visualizar métricas relevantes sobre el uso del chatbot, con el objetivo de proporcionar información útil para la toma de decisiones y el monitoreo del comportamiento de los usuarios.

Este dashboard fue implementado como una sección independiente dentro de la aplicación (/analytics), desarrollada en Next.js e integrada con Supabase para la consulta de datos en tiempo real. La interfaz presenta indicadores clave (KPIs) como número de usuarios registrados, usuarios activos, cantidad de conversaciones, total de mensajes, preguntas realizadas y promedio de interacción por sesión.

El acceso al dashboard está restringido únicamente a usuarios con rol de **superUser**, lo que garantiza que la información analítica sea visible solo para perfiles autorizados. Esta validación se realiza en el frontend al momento de cargar la vista, redirigiendo automáticamente a usuarios no autorizados. Este enfoque permite mantener la seguridad de los datos y separar claramente las funcionalidades del sistema entre usuarios finales y administradores.

Visualización de métricas

La interfaz incluye diferentes componentes visuales diseñados para facilitar la interpretación de los datos:

- **Tarjetas de resumen de KPIs:** muestran valores clave de forma interactiva
- **Gráficas dinámicas:** representan la evolución de mensajes, conversaciones y usuarios activos a lo largo del tiempo
- **Ranking de usuarios y preguntas frecuentes:** permite identificar patrones de uso y temas más consultados
- **Filtros por periodo:** permiten analizar los datos en distintos rangos de tiempo

Asimismo, cada métrica puede ser explorada en detalle mediante modales interactivos, y los datos pueden ser exportados en formato CSV para su análisis externo.

Valor analítico

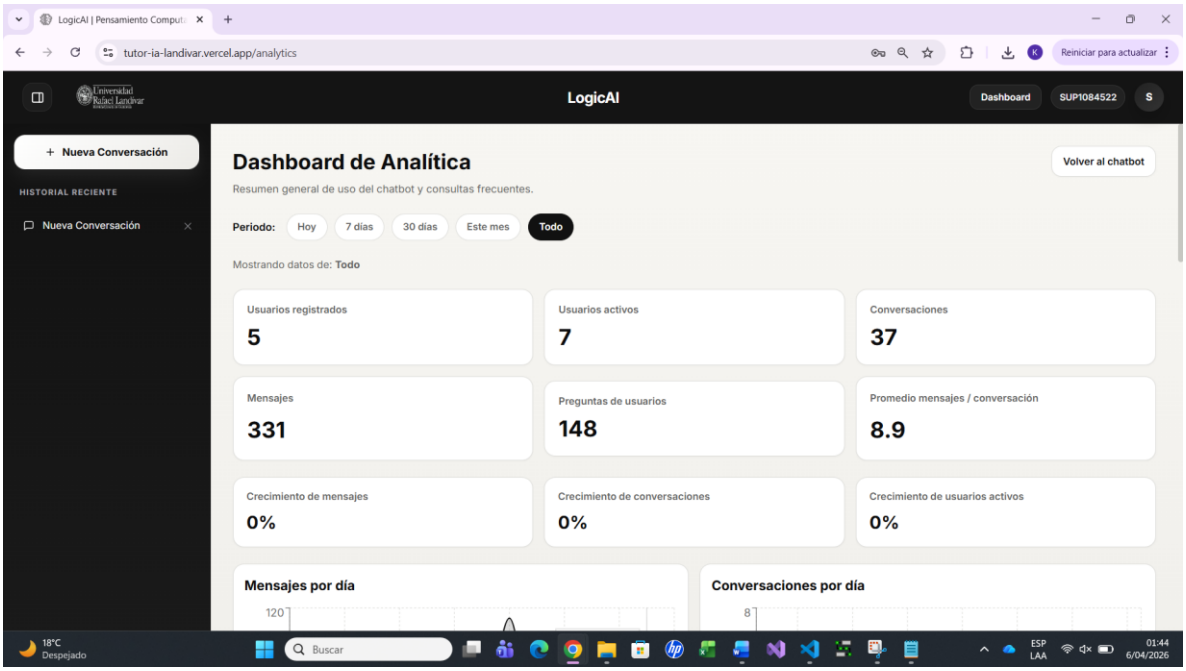
- Permite monitorear el uso del chatbot en tiempo real
- Identifica tendencias de comportamiento de los usuarios
- Evalúa la efectividad del sistema
- Detecta oportunidades de mejora
- El dashboard no solo actúa como una herramienta visual, sino como un componente estratégico para el análisis y evolución del sistema.

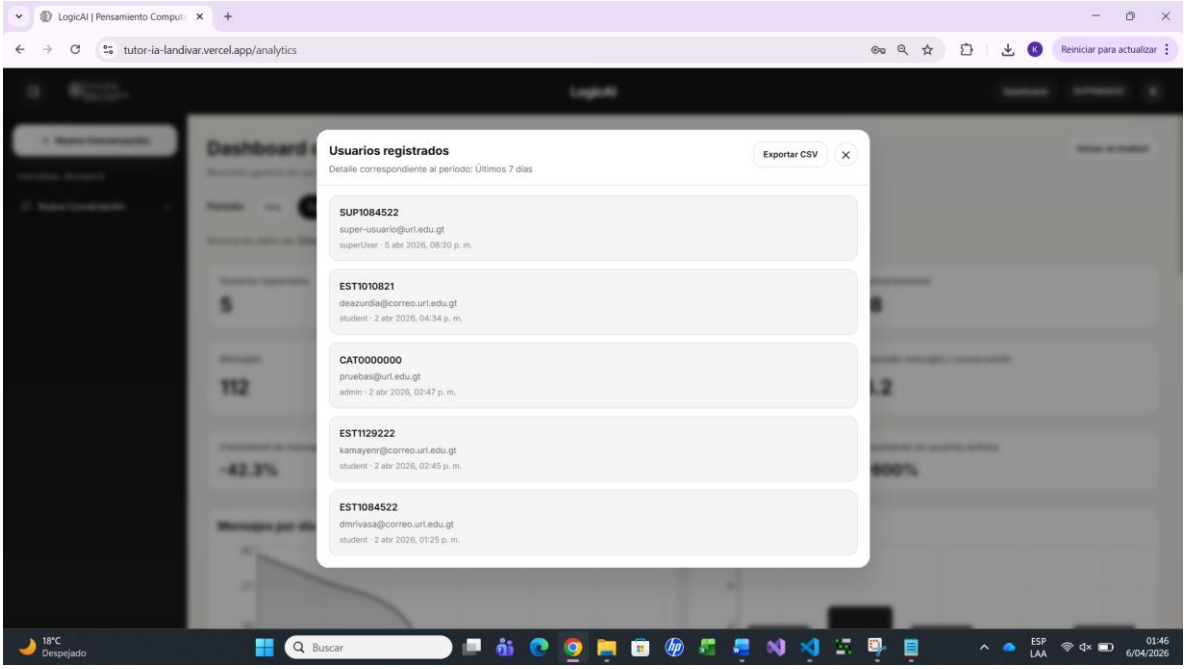
Coherencia relevancia de los datos

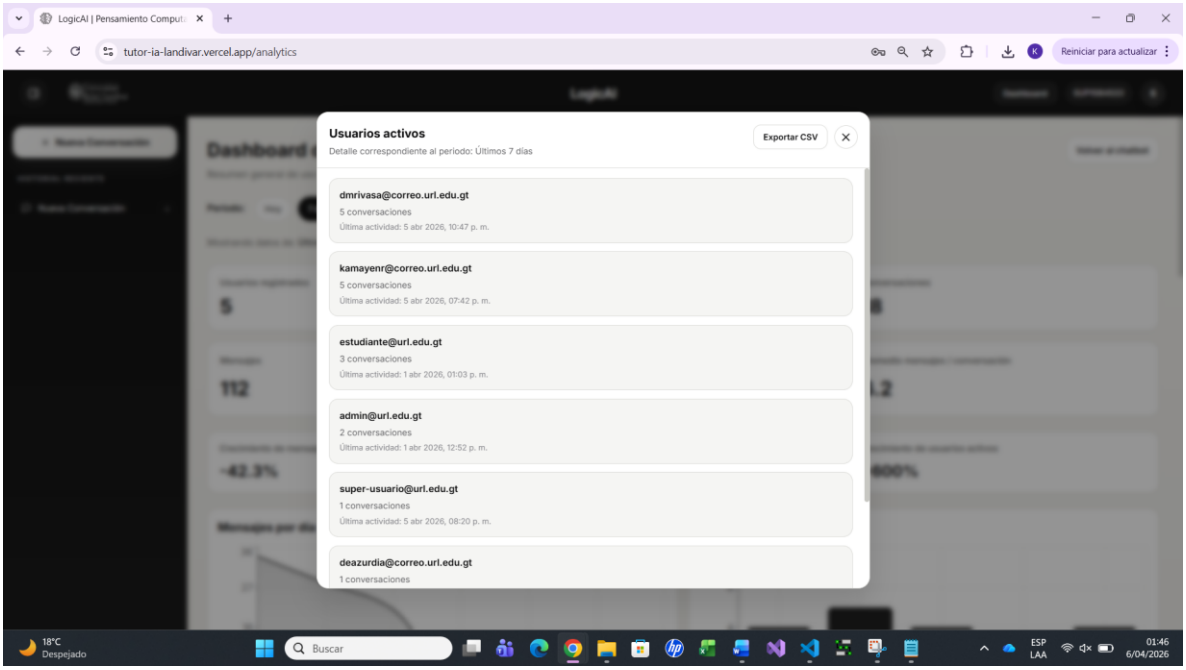
Las métricas presentadas en el dashboard se construyen a partir de datos reales almacenados en la base de datos del sistema, específicamente en las tablas de usuarios, sesiones y mensajes. El procesamiento incluye agrupaciones, conteos y normalización de datos, permitiendo generar información coherente y significativa.

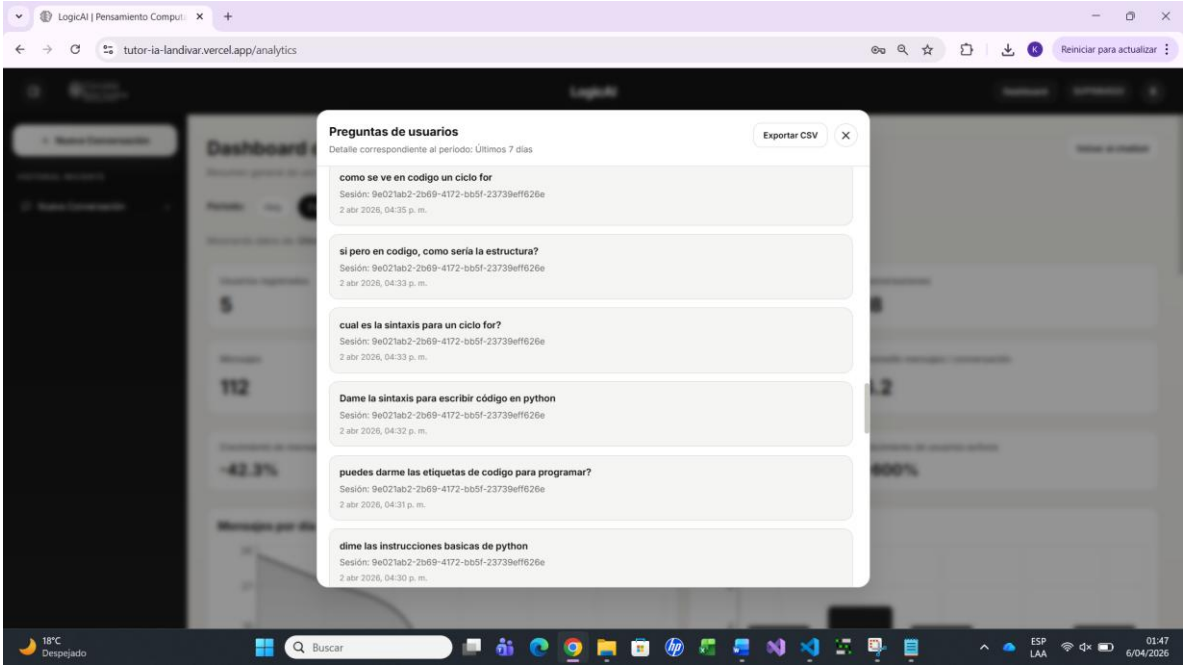
Se implementaron mecanismos de filtrado por fechas y cálculo de crecimiento, lo que permite contextualizar los datos y evitar interpretaciones aisladas.

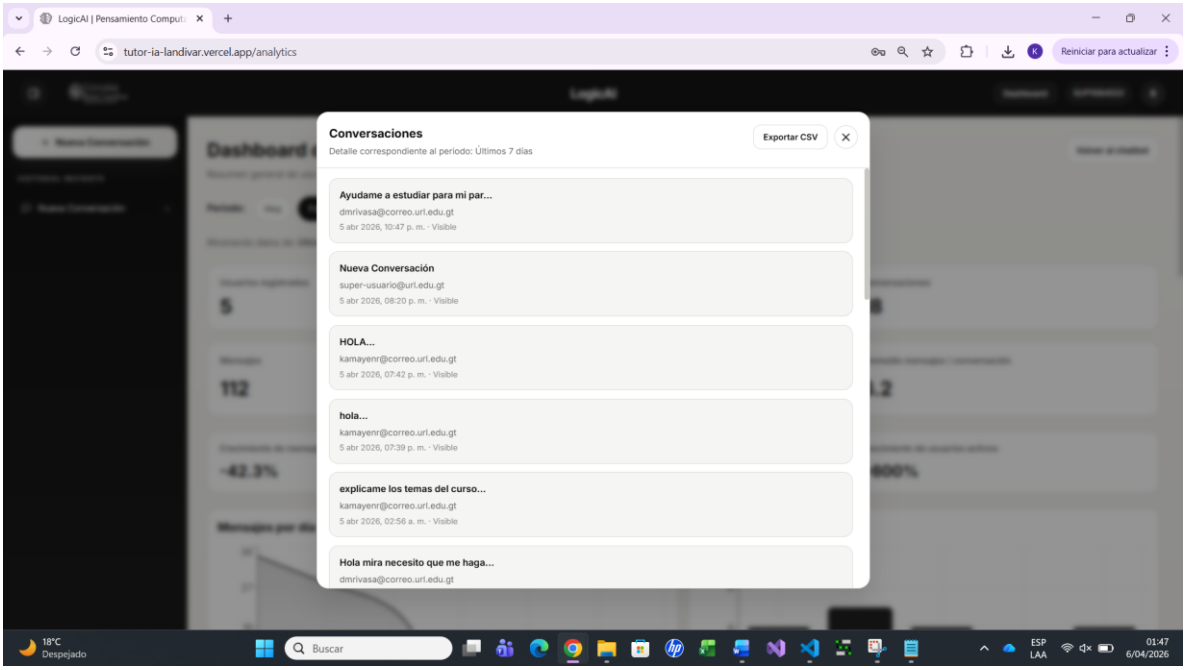
Dashboard visual en el sistema

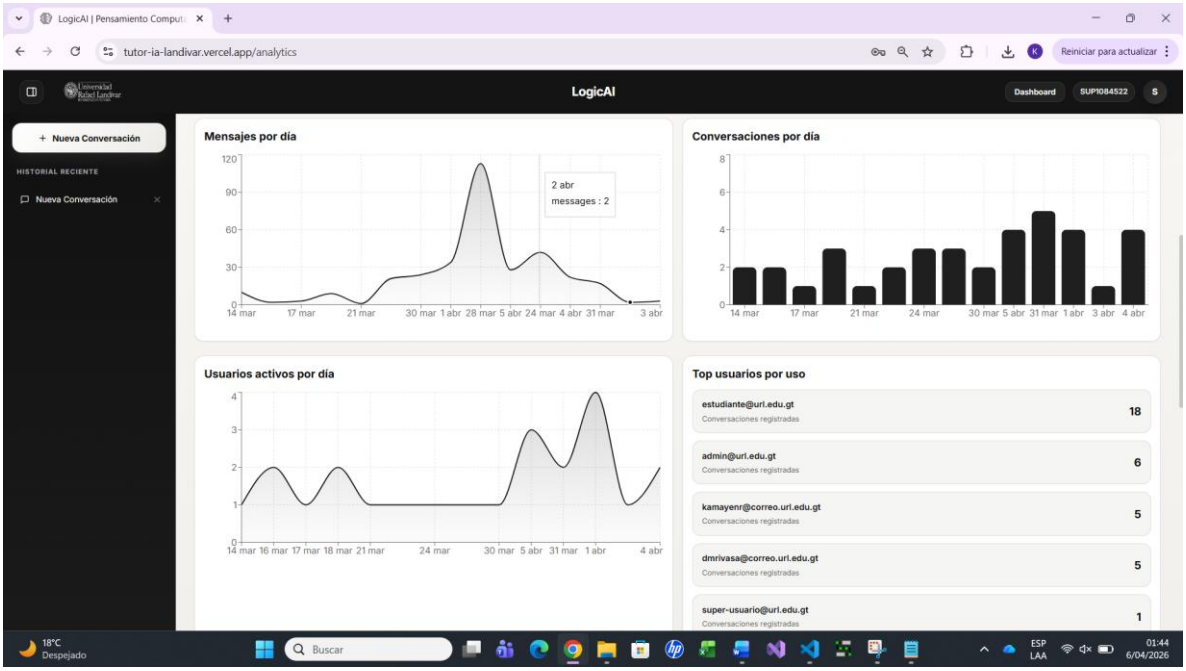


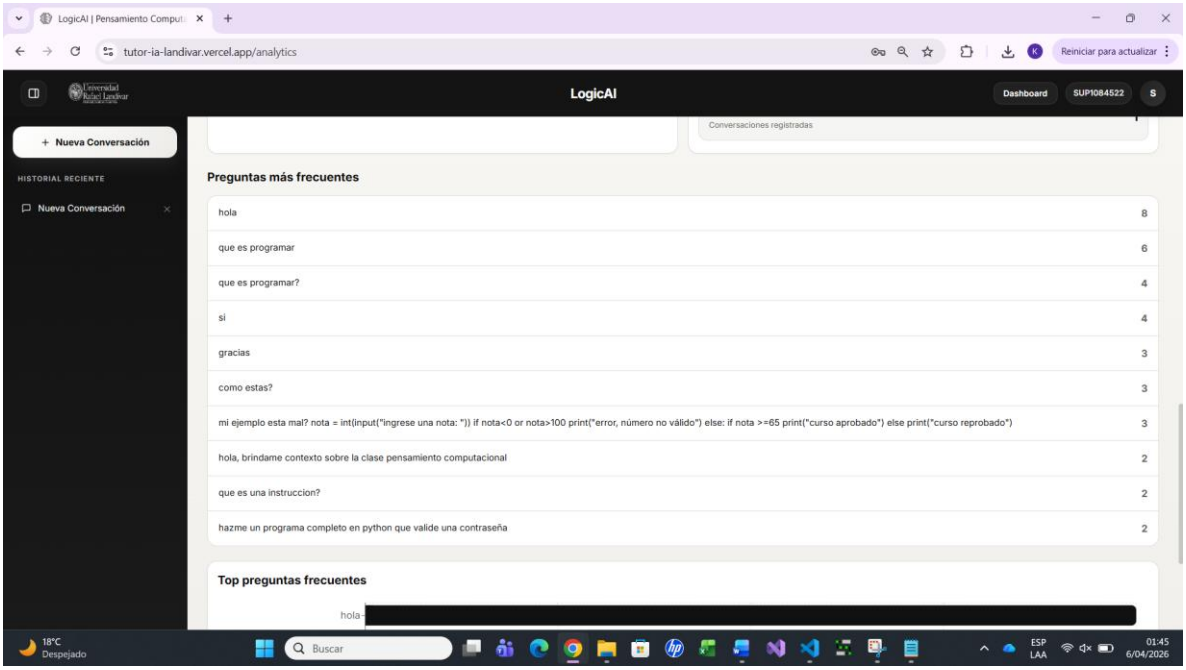

LogicAI | Pensamiento Computa
tutor-ia-landivar.vercel.app/analytics
Reiniciar para actualizar
Usuarios registrados
Detalle correspondiente al periodo: Últimos 7 días
Exportar CSV
SUP1084522
super-usuario@url.edu.gt
superUser · 5 abr 2026, 08:20 p. m.
EST1010821
deazurdia@correo.url.edu.gt
student · 2 abr 2026, 04:34 p. m.
CAT0000000
pruebas@url.edu.gt
admin · 2 abr 2026, 02:47 p. m.
EST1129222
kamayenr@correo.url.edu.gt
student · 2 abr 2026, 02:45 p. m.
EST1084522
dmrivasa@correo.url.edu.gt
student · 2 abr 2026, 01:25 p. m.
18°C
Despejado
Buscar
ESP
LAA
01:46
6/04/2026

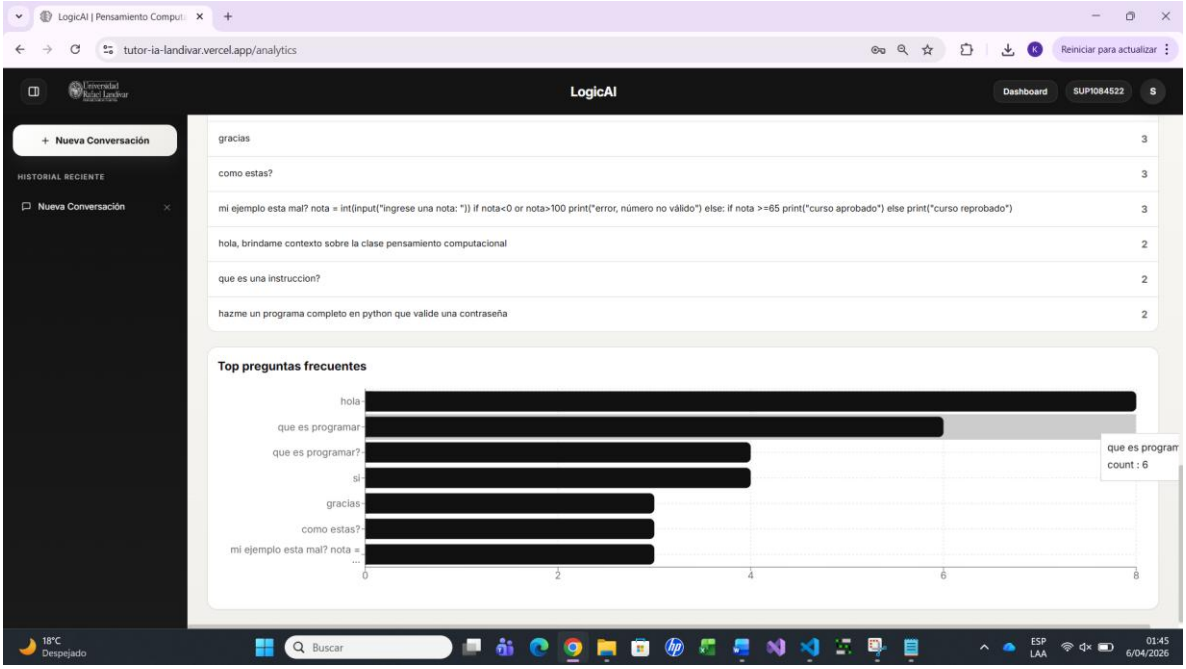

LogicAI | Pensamiento Computa
tutor-ia-landivar.vercel.app/analytics
Reiniciar para actualizar
Usuarios activos
Detalle correspondiente al periodo: Últimos 7 días
Exportar CSV
dmrivasa@correo.url.edu.gt
5 conversaciones
Última actividad: 5 abr 2026, 10:47 p. m.
kamayenr@correo.url.edu.gt
5 conversaciones
Última actividad: 5 abr 2026, 07:42 p. m.
estudiante@url.edu.gt
3 conversaciones
Última actividad: 1 abr 2026, 01:03 p. m.
admin@url.edu.gt
2 conversaciones
Última actividad: 1 abr 2026, 12:52 p. m.
super-usuario@url.edu.gt
1 conversaciones
Última actividad: 5 abr 2026, 08:20 p. m.
deazurdia@correo.url.edu.gt
1 conversaciones
18°C
Despejado
Buscar
ESP
LAA
01:46
6/04/2026


LogicAI | Pensamiento Computa
tutor-ia-landivar.vercel.app/analytics
Reiniciar para actualizar
Preguntas de usuarios
Detalle correspondiente al período: Últimos 7 días
Exportar CSV
como se ve en codigo un ciclo for
Sesión: 9e021ab2-2b69-4172-bb5f-23739eff626e
2 abr 2026, 04:35 p. m.
si pero en codigo, como sería la estructura?
Sesión: 9e021ab2-2b69-4172-bb5f-23739eff626e
2 abr 2026, 04:33 p. m.
cual es la sintaxis para un ciclo for?
Sesión: 9e021ab2-2b69-4172-bb5f-23739eff626e
2 abr 2026, 04:33 p. m.
Dame la sintaxis para escribir código en python
Sesión: 9e021ab2-2b69-4172-bb5f-23739eff626e
2 abr 2026, 04:32 p. m.
puedes darme las etiquetas de codigo para programar?
Sesión: 9e021ab2-2b69-4172-bb5f-23739eff626e
2 abr 2026, 04:31 p. m.
dime las instrucciones basicas de python
Sesión: 9e021ab2-2b69-4172-bb5f-23739eff626e
2 abr 2026, 04:30 p. m.
18°C
Despejado
Buscar
ESP
LAA
01:47
6/04/2026


LogicAI | Pensamiento Computa
tutor-ia-landivar.vercel.app/analytics
Reiniciar para actualizar
Conversaciones
Detalle correspondiente al período: Últimos 7 días
Exportar CSV
Ayudame a estudiar para mi par...
dmrivasa@correo.url.edu.gt
5 abr 2026, 10:47 p. m. · Visible
Nueva Conversación
super-usuario@url.edu.gt
5 abr 2026, 08:20 p. m. · Visible
HOLA...
kamayenr@correo.url.edu.gt
5 abr 2026, 07:42 p. m. · Visible
hola...
kamayenr@correo.url.edu.gt
5 abr 2026, 07:39 p. m. · Visible
explicame los temas del curso...
kamayenr@correo.url.edu.gt
5 abr 2026, 02:56 a. m. · Visible
Hola mira necesito que me haga...
dmrivasa@correo.url.edu.gt
18°C
Despejado
Buscar
ESP
LAA
01:47
6/04/2026


LogicAI | Pensamiento Comput
tutor-ia-landivar.vercel.app/analytics
Reiniciar para actualizar
LogicAI
Dashboard
SUP1084522
+ Nueva Conversación
HISTORIAL RECIENTE
Nueva Conversación
Mensajes por día
2 abr
messages : 2
Conversaciones por día
Usuarios activos por día
Top usuarios por uso
estudiante@url.edu.gt
Conversaciones registradas
18
admin@url.edu.gt
Conversaciones registradas
6
kamayenr@correo.url.edu.gt
Conversaciones registradas
5
dmrivasa@correo.url.edu.gt
Conversaciones registradas
5
super-usuario@url.edu.gt
Conversaciones registradas
1
18°C
Despejado
Buscar
ESP
LAA
01:44
6/04/2026


LogicAI | Pensamiento Comput
tutor-ia-landivar.vercel.app/analytics
Reiniciar para actualizar
LogicAI
Dashboard
SUP1084522
+ Nueva Conversación
HISTORIAL RECIENTE
Nueva Conversación
Conversaciones registradas
Preguntas más frecuentes
hola 8
que es programar 6
que es programar? 4
si 4
gracias 3
como estas? 3
mi ejemplo esta mal? nota = int(input("ingrese una nota: ")) if nota<0 or nota>100 print("error, número no válido") else: if nota >=65 print("curso aprobado") else print("curso reprobado") 3
hola, brindame contexto sobre la clase pensamiento computacional 2
que es una instruccion? 2
hazme un programa completo en python que valide una contraseña 2
Top preguntas frecuentes
hola
18°C
Despejado
Buscar
ESP
LAA
01:45
6/04/2026

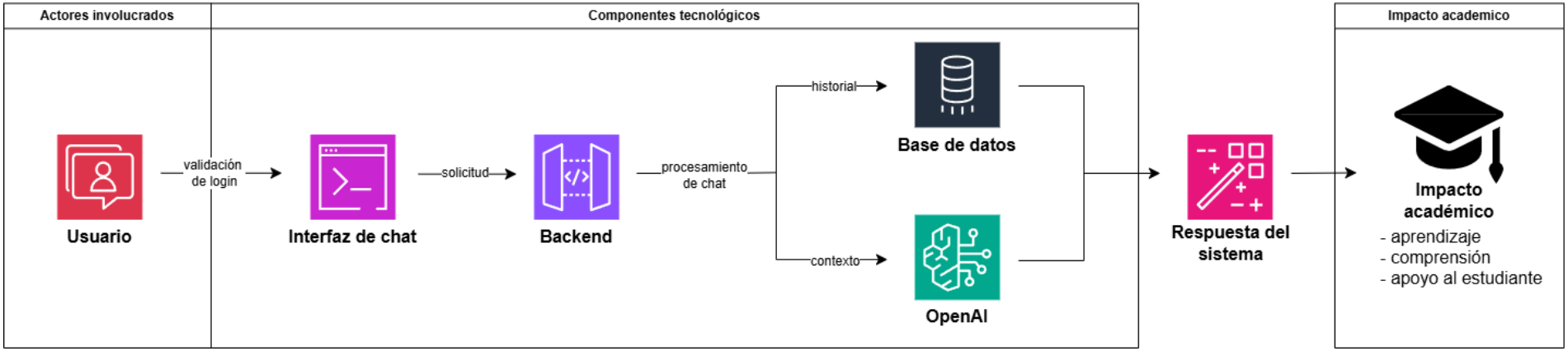

LogicAI | Pensamiento Compu
tutor-ia-landivar.vercel.app/analytics
Reiniciar para actualizar
LogicAI
Dashboard
SUP1084522
+ Nueva Conversación
HISTORIAL RECIENTE
Nueva Conversación
gracias 3
como estas? 3
mi ejemplo esta mal? nota = int(input("ingrese una nota: ")) if nota<0 or nota>100 print("error, número no válido") else: if nota >=65 print("curso aprobado") else print("curso reprobado") 3
hola, brindame contexto sobre la clase pensamiento computacional 2
que es una instruccion? 2
hazme un programa completo en python que valide una contraseña 2
Top preguntas frecuentes
hola
que es programar
que es programar?
si
gracias
como estas?
mi ejemplo esta mal? nota = ...
0
2
4
6
8
que es program
count : 6
18°C
Despejado
Buscar
ESP
LAA
01:45
6/04/2026

Mapa del sistema

En este modelo, el usuario interactúa con la interfaz de chat, desde donde se envían solicitudes al backend del sistema. Este backend procesa la información, consulta el historial almacenado en la base de datos y se comunica con el modelo de inteligencia artificial para generar una respuesta.

El flujo de información permite mantener el contexto de la conversación y ofrecer respuestas coherentes. El sistema impacta directamente en el proceso académico, al proporcionar apoyo personalizado, fomentar la comprensión conceptual y facilitar el aprendizaje autónomo del estudiante.

Además del usuario estudiante, el sistema contempla el rol de superUser, el cual tiene acceso a funcionalidades administrativas específicas, como el dashboard de analítica. Este rol permite supervisar el uso del sistema, consultar métricas relevantes y analizar el comportamiento de los usuarios dentro de la plataforma.



Dataset de fine-tuning

El proceso de fine-tuning del modelo se basó en un dataset estructurado en formato JSONL, diseñado para representar interacciones entre un estudiante y un tutor académico. Cada entrada del dataset corresponde a una conversación, compuesta por una secuencia de mensajes con roles definidos.

Cada registro incluye un arreglo de mensajes donde se identifican los roles:

- **system:** representado el contexto de cómo debe actuar el agente
- **user:** representando la consulta del estudiante.
- **assistant:** representando la respuesta del tutor.

```
{
    "messages": [
        {
            "role": "system",
            "content": "Eres un tutor académico del curso Pensamiento Computacional. Explicas conceptos de forma clara, pedagógica y paso a paso.
                        Ayudas a razonar ejercicios y a comprender código, pero no entregas soluciones completas listas para ejecutar. Puedes dar
                        orientación, pasos, ejemplos pequeños y pseudocódigo breve cuando sea útil."
        },

        {
            "role": "user",
            "content": "¿Qué es programar?"
        },

        {
            "role": "assistant",
            "content": "Programar es dar instrucciones a la computadora con el objetivo de que realice tareas. La computadora está lista para
                        ejecutar instrucciones precisas que sean brindadas a través de un lenguaje de programación.\n\nEn resumen: programar
                        es comunicarle a la computadora, en un lenguaje que ella entienda, exactamente qué pasos debe seguir para resolver un
                        problema."
        }
    ]
}
```

Estructura y contenido del dataset

El dataset fue diseñado para reflejar el comportamiento deseado del tutor, incluyendo:

- preguntas relacionadas con el curso
- respuestas explicativas, guiadas y paso a paso
- restricciones para evitar la generación de código completo
- ejemplos de seguimiento de conversación

Esto permite que el modelo aprenda no solo qué responder, sino también cómo responder, manteniendo un enfoque educativo.

División del dataset (train y validation)

Para el proceso de fine-tuning, el dataset fue dividido en dos conjuntos:

- **Training dataset (train)**: utilizado para entrenar el modelo.
- **Validation dataset (valid)**: utilizado para evaluar el desempeño del modelo durante el entrenamiento.

Esta separación es importante porque permite medir la capacidad del modelo para generalizar y responder correctamente a ejemplos que no ha visto durante el entrenamiento. De esta manera, se evita el sobreajuste y se asegura una mejor calidad en las respuestas generadas.

Validación del dataset

Antes de ser utilizado en el proceso de fine-tuning, el dataset fue validado mediante un script automatizado que verifica que cada entrada cumpla con la estructura requerida, incluyendo la presencia de los campos role y content, así como la correcta organización de los mensajes.